# 6

# Clase 6 - Análisis sintáctico

## 6.1 Analizador sintáctico

El analizador sintáctico consulta la tabla de símbolos para obtener información adicional sobre el token que le pasó el analizador léxico; no suele acceder a la información específica del lexema. En ocasiones, también puede modificar la tabla de símbolos.

En un compilador, el analizador sintáctico es independiente, pero trabaja en equipo con los analizadores léxico y semántico. El analizador sintáctico se enfoca en resolver problemas de frases, mientras que el léxico se encarga de los problemas de palabras.

> **Nota 6.1.1** Observación
>
> Puede haber palabras "compuestas", en las que varias palabras forman un lexema.

Los conceptos de derivaciones, producciones, etc., son iguales que en un TDS (traductor dirigido por la sintaxis).

## 6.2 Eliminación de la recursión por la izquierda

### 6.2.1 Algoritmo del Libro Sección 4.19:

```
1)   ordenar los no terminales de cierta forma A_1, A_2, ..., A_n
2)   for (cada i de 1 a n) {
3)       for (cada j de 1 a i − 1) {
4)           sustituir cada produccion de la forma A_i ->A_j γ por las
             producciones A_i ->δ_1 γ | δ_2 γ | ... | δ_k γ en donde A_j ->δ_1 | δ_2 | ... | δ_k sean todas las
             producciones de A_j actuales
5)       }
6)       eliminar la recursividad inmediata por la izquierda entre las producciones A_i
7)   }
```

Este algoritmo elimina la recursión por la izquierda **directa e indirecta** en una gramática.

**Nota 6.2.1** Restricciones de aplicación

Antes de aplicar el algoritmo, deben eliminarse las producciones vacías y los ciclos de la gramática de entrada. Además, en las alternativas $A \to A\gamma$ se exige $\gamma \neq \varepsilon$. Los ejemplos y la explicación se desarrollan en la sección 6.3.3.

### 6.2.2 Elección del orden de los no terminales

Una gramática en BNF es un conjunto de producciones: el orden en que se escriben no impone un orden de aplicación. La línea 1 del algoritmo establece un orden de procesamiento para los no terminales, que no tiene por qué coincidir con el orden en que aparecen en el BNF.

Ordenar "de cierta forma" significa elegir el orden de los no terminales que convenga para simplificar la aplicación del algoritmo.

**Nota 6.2.2** Recursividad inmediata

La recursividad inmediata es la recursividad directa, es decir, una producción de la forma $A \to A\alpha$, que ya conocemos.

El primer `for` recorre los no terminales de la gramática. El segundo recorre los no terminales anteriores en el orden elegido. Para cada producción de $A_i$ cuyo cuerpo comienza por $A_j$, con $j < i$, sustituimos ese $A_j$ por los cuerpos de sus producciones actuales.

## 6.3 Ejemplo de aplicación del algoritmo

Como se trata de la BNF de un analizador sintáctico, los símbolos terminales deberían ser tokens; sin embargo, para simplificar el ejemplo, se utilizan letras:

$$S \to A\,a \mid b,$$
$$A \to A\,c \mid S\,d \mid \varepsilon.$$

### 6.3.1 Primer orden: S, A

Ordenamos los no terminales de la siguiente manera:

1) $S \to A\,a \mid b.$
2) $A \to A\,c \mid S\,d \mid \varepsilon.$

**Primera iteración**

En el primer `for`, $i = 1$ y $A_1 = S$. El segundo `for` no se ejecuta, porque $i - 1 = 0$: no hay producciones para sustituir. Pasamos al siguiente valor de $i$.

**Segunda iteración**

En el primer `for`, $i = 2$ y $A_2 = A$. En el segundo, $j = 1$: sustituimos $S$ en $A \to A\,c \mid S\,d \mid \varepsilon$ por $S \to A\,a \mid b$. La producción se transforma en:

$$A \to A\,c \mid (A\,a \mid b)\,d \mid \varepsilon,$$
$$A \to A\,c \mid A\,a\,d \mid b\,d \mid \varepsilon.$$

Ahora eliminamos la recursividad directa por la izquierda en $A$:

$$A \to b\,d\,R \mid R,$$
$$R \to c\,R \mid a\,d\,R \mid \varepsilon.$$

**Gramática resultante**

Finalmente, la gramática equivalente sin recursión por la izquierda es:

$$S \to A\,a \mid b,$$
$$A \to b\,d\,R \mid R,$$
$$R \to c\,R \mid a\,d\,R \mid \varepsilon.$$

> **Nota 6.3.1** Gramática predictiva
>
> Esto no significa que la gramática sea predictiva, ya que solamente se eliminó la recursión por la izquierda.

## 6.3.2 Orden inverso: A, S

Invertimos el orden:

1) $A \to A\,c \mid S\,d \mid \varepsilon.$
2) $S \to A\,a \mid b.$

**Primera iteración**

Para $i = 1$, el segundo `for` no se ejecuta. Eliminamos la recursividad directa por la izquierda en $A$:

$$A \to S\,d\,R \mid R,$$
$$R \to c\,R \mid \varepsilon.$$

**Segunda iteración**

Para $i = 2$ y $j = 1$, sustituimos $A$ en el lado derecho de $S$, utilizando la producción de $A$ cuya recursividad directa por la izquierda ya fue eliminada:

$$S \to (S\,d\,R \mid R)\,a \mid b,$$
$$S \to S\,d\,R\,a \mid R\,a \mid b.$$

Ahora eliminamos la recursividad directa por la izquierda en $S$:

$$S \to R\,a\,R'' \mid b\,R'',$$
$$R'' \to d\,R\,a\,R'' \mid \varepsilon.$$

**Gramática resultante**

Finalmente, la gramática queda:

$$S \to R\,a\,R'' \mid b\,R'',$$
$$R'' \to d\,R\,a\,R'' \mid \varepsilon,$$
$$A \to S\,d\,R \mid R,$$
$$R \to c\,R \mid \varepsilon.$$

> **Nota 6.3.2** Observación sobre el orden
>
> Si tenemos $n$ no terminales, tenemos $n!$ formas de ordenarlos. Cada orden puede dar un resultado distinto y una complejidad de cálculo diferente; por eso es importante elegir un orden que nos convenga.

> **Consejo 6.3.1** Recomendación del profesor
>
> Analizar la gramática para evitar generar muchas producciones al eliminar la recursión directa por la izquierda, lo que produce muchas reglas de tipo $R$. Por lo tanto, **no elegir primero los no terminales que tengan recursión directa por la izquierda**. Así, posiblemente podamos eliminar la recursión indirecta y directa en una sola pasada.

## 6.3.3 Nota adicional sobre la aplicación del algoritmo

**Pregunta 6.3.1** ¿Hay que considerar la R creada por la eliminación de la recursión por la izquierda como un nuevo no terminal e iterar el algoritmo hasta $i = 3$?

**Respuesta:** Hay que considerar $R$ en los casos con un doble patrón y una alternativa vacía, como en la producción

$$A \to A\,A\,a \mid \varepsilon.$$

Al eliminar la recursión por la izquierda, se convierte en:

$$A \to R,$$
$$R \to A\,a\,R \mid \varepsilon.$$

Esta gramática aún tiene recursión indirecta por la izquierda.
Se debe prestar especial atención cuando hay una alternativa vacía en la producción, ya que puede generar recursividad indirecta por la izquierda. Si no hay una alternativa vacía, no es necesario considerar $R$, porque se introduce un término $\beta$ que evita esa recursividad.
Por ejemplo:

$$A \to A\,A\,a \mid b.$$

Al eliminar la recursión directa, queda:

$$A \to b\,R,$$
$$R \to A\,a\,R \mid \varepsilon.$$

**El verdadero problema puede ocurrir cuando hay producciones que generan la cadena vacía.**

**Otros casos que pueden hacer fallar el algoritmo**

Además del ejemplo anterior, deben considerarse los siguientes casos.[1]

**Ciclos entre no terminales**

$$A_1 \to A_2 \mid a,$$
$$A_2 \to A_1 \mid b.$$

El ciclo $A_1 \Rightarrow A_2 \Rightarrow A_1$ puede producir una regla $A_2 \to A_2$ al sustituir. Correspondería a $\alpha = \varepsilon$, fuera de la transformación directa. Eliminando el ciclo, ambas variables pueden tener las alternativas $a \mid b$.

**Recursión oculta por producciones vacías**

$$A_3 \to A_2 A_1,$$
$$A_2 \to \varepsilon,$$
$$A_1 \to A_2 A_1.$$

Aunque $A_1 \to A_2 A_1$ no comienza por $A_1$, existe la derivación $A_1 \Rightarrow A_2 A_1 \Rightarrow A_1$. Las sustituciones que solo examinan el primer símbolo pueden no eliminar esta recursión.

**Anulabilidad indirecta** No basta con buscar alternativas escritas como $A \to \varepsilon$; también puede ocurrir $A \Rightarrow^* \varepsilon$ a través de otros no terminales.

[1] Referencia: Marc Moreno Maza, Elimination of left recursion.

**Nota 6.3.3** Preparación de la gramática y producciones vacías

Antes de aplicar el algoritmo para eliminar la recursión por la izquierda, se deben:

- Eliminar las producciones vacías, de la forma $A \to \varepsilon$.
- Eliminar los ciclos, es decir, las derivaciones que vuelven al mismo no terminal después de uno o más pasos: $A \Rightarrow^+ A$.

Estos pasos deben conservar el lenguaje de la gramática, incluida la cadena vacía si pertenece a él.
Al eliminar la recursión directa, la transformación introduce nuevas producciones vacías, como $R \to \varepsilon$. Esto es parte del procedimiento: si se realizaron los pasos previos, el algoritmo no vuelve a introducir ciclos.